第8章 指针

8.1 学习目标

- 理解指针的概念和内存地址
- 掌握指针变量的定义和运算
- 理解指针与数组的关系
- 掌握指针在函数中的应用
- 了解动态内存分配

8.2 为什么需要指针

```c
// 问题1：函数参数是值传递，如何在函数内修改外部变量？
void modify(int *p)     // 传地址
{
    *p = 100;           // 通过指针修改原变量
}

// 问题2：数组在传参时退化为指针
void printArray(int *arr, int n);

// 问题3：动态分配内存
int *p = malloc(100 * sizeof(int));  // 运行时分配内存
```

> 指针是C语言最强大的特性，也是理解C语言底层机制的关键。

8.3 指针的基本概念

8.3.1 什么是指针

> 指针 = 存放内存地址的变量

```
内存示意（每个格子是一个字节）：
地址:    0x1000   0x1004   0x1008
         ┌──────┬──────┬──────┐
内容:    |  10  |  20  | ...  |
         └──────┴──────┴──────┘
变量:       a        b

int a = 10;
int *p = &a;    // p 存储了 a 的地址 0x1000
```

8.3.2 取地址运算符 `&` 和间接访问运算符 `*`

```c
#include <stdio.h>
int main()
{
    int a = 10;

    int *p = &a;      // &a: 取变量a的地址
                      // p 是指针变量，存储a的地址

    printf("a的值: %d\n", a);        // 10
    printf("a的地址: %p\n", &a);      // 如: 0x7ffd1234abcd
    printf("p的值(即a的地址): %p\n", p);
    printf("*p的值(即a的值): %d\n", *p);  // 10

    *p = 20;           // 通过指针修改变量
    printf("修改后a的值: %d\n", a); // 20

    return 0;
}
```

8.3.3 指针变量的大小

```c
printf("sizeof(char*) = %d\n", sizeof(char*));    // 4(32位)或8(64位)
printf("sizeof(int*)  = %d\n", sizeof(int*));
printf("sizeof(double*)= %d\n", sizeof(double*));
// 所有类型的指针变量大小相同（因为都存地址）
```

8.4 指针的运算

8.4.1 指针与整数的加减

```c
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;     // p指向arr[0]

    printf("*p = %d\n", *p);         // 10
    printf("*(p+1) = %d\n", *(p+1)); // 20
    printf("*(p+2) = %d\n", *(p+2)); // 30

    p++;      // p移动到下一个int，地址实际加了sizeof(int)=4
    printf("*p = %d\n", *p);         // 20

    return 0;
}
```

> **关键**：`p + n` 的实际地址偏移 = `p` 的地址 + `n × sizeof(指向的类型)`。

8.4.2 指针的比较

```
int *p = &arr[0];
int *q = &arr[3];

if (p < q)  // 比较地址大小
    printf("p指向的元素在q之前\n");
```

8.4.3 指针的差值

```
int *p = &arr[0];
int *q = &arr[4];

printf("相差元素个数: %d\n", q - p);  // 4
```

8.5 指针与数组

8.5.1 数组名即指针

```
int arr[5] = {1, 2, 3, 4, 5};

// 以下等价
arr[2] = 100;       // 下标访问
*(arr + 2) = 100;   // 指针访问

// arr 是常量指针，不能修改
// arr++;   ? 错误！数组名是地址常量
int *p = arr;
p++;            // ? 指针变量可以修改
```

8.5.2 用指针遍历数组

```
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);

    // 方法1：指针移动法
    int *p = arr;
    for (int i = 0; i < n; i++)
    {
        printf("%d ", *p);
        p++;
    }
    printf("\n");

    // 方法2：基址+偏移（最常用）
```

```
    for (int i = 0; i < n; i++)
        printf("%d ", *(arr + i));
    printf("\n");

    // 方法3：下标法（本质也是指针运算）
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

8.5.3 指针与二维数组

```
int matrix[3][4] = {
    {1,  2,  3,  4},
    {5,  6,  7,  8},
    {9, 10, 11, 12}
};

// 访问方式
matrix[1][2] == 7;            // 下标访问
*(*(matrix + 1) + 2) == 7;    // 指针访问
*(matrix[1] + 2) == 7;        // 混合访问
```

8.6 指针与函数

8.6.1 指针作为函数参数（实现"引用传递"）

```
#include <stdio.h>

// 通过指针实现真正的交换
void swap(int *a, int *b)
{
    int temp = *a;    // 取出a指向的值
    *a = *b;          // 把b指向的值赋给a指向的位置
    *b = temp;
}

int main()
{
    int x = 10, y = 20;
    printf("交换前: x=%d, y=%d\n", x, y);
    swap(&x, &y);     // 传递x和y的地址
    printf("交换后: x=%d, y=%d\n", x, y);  // x=20, y=10 ?
    return 0;
}
```

8.6.2 函数返回指针

```c
// 返回两个数中较大的地址
int* maxAddr(int *a, int *b)
{
    return (*a > *b) ? a : b;
}

int main()
{
    int x = 10, y = 20;
    int *pmax = maxAddr(&x, &y);
    printf("较大值: %d\n", *pmax);  // 20
    // 通过返回的指针修改原变量
    *pmax = 999;
    printf("x=%d, y=%d\n", x, y);   // x=10, y=999
    return 0;
}
```

> **警告**：不要返回局部变量的地址！

```c
// ? 错误示例
int* badFunction()
{
    int local = 10;
    return &local;   // 函数返回后local已销毁，返回的地址无效!
}
```

8.7 指针与字符串

```c
#include <stdio.h>
#include <string.h>

int main()
{
    // 方式1：字符数组
    char str1[] = "Hello";
    str1[0] = 'h';        // ? 可以修改
    printf("%s\n", str1); // "hello"

    // 方式2：字符串常量指针
    char *str2 = "World";
    // str2[0] = 'w';      // ? 错误! 常量字符串不能修改
    printf("%s\n", str2);  // "World"

    // 用指针遍历字符串
    char *p = str1;
    while (*p != '\0')
    {
        printf("%c ", *p);
        p++;
    }
    return 0;
```

```
}
```

字符串处理函数的指针实现

```
// 自定义 strlen
int myStrlen(const char *str)
{
    const char *p = str;
    while (*p != '\0')
        p++;
    return p - str;   // 指针差值 = 字符个数
}

// 自定义 strcpy
char* myStrcpy(char *dest, const char *src)
{
    char *p = dest;
    while ((*dest++ = *src++) != '\0')
        ;
    return p;
}
```

8.8 指针数组与数组指针

8.8.1 指针数组

```
// 指针数组：每个元素都是指针
int a = 1, b = 2, c = 3;
int *arr[3] = {&a, &b, &c};   // 三个int指针组成的数组

for (int i = 0; i < 3; i++)
    printf("%d ", *arr[i]);    // 输出: 1 2 3
```

```
// 应用：存储多个字符串
char *names[] = {"Alice", "Bob", "Charlie", "David"};
for (int i = 0; i < 4; i++)
    printf("%s\n", names[i]);
```

8.8.2 数组指针

```
// 数组指针：指向数组的指针
int a[3] = {1, 2, 3};
int (*p)[3] = &a;   // p是指向长度为3的int数组的指针

printf("%d\n", (*p)[0]);  // 1
printf("%d\n", (*p)[1]);  // 2
```

8.8.3 区分

```
int *p[5];     // 指针数组：p是包含5个int*的数组（[]优先级高）
int (*p)[5];   // 数组指针：p是指向长度为5的int数组的指针
```

8.9 动态内存分配

8.9.1 四个核心函数（stdlib.h）

函数	功能
`malloc(size)`	分配 size 字节，内容未初始化
`calloc(n, size)`	分配 n×size 字节，初始化为0
`realloc(ptr, size)`	重新调整已分配的内存大小
`free(ptr)`	释放已分配的内存

8.9.2 基本用法

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n;
    printf("请输入学生数: ");
    scanf("%d", &n);

    // 动态分配数组
    int *scores = (int*)malloc(n * sizeof(int));
    if (scores == NULL)
    {
        printf("内存分配失败！\n");
        return 1;
    }

    // 输入成绩
    for (int i = 0; i < n; i++)
    {
        printf("第%d个学生成绩: ", i + 1);
        scanf("%d", &scores[i]);   // 和普通数组一样使用
    }

    // 计算平均分
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += scores[i];
    printf("平均分: %.2f\n", (float)sum / n);
```

```
    // 释放内存
    free(scores);
    // scores = NULL;  // 好习惯：防止野指针

    return 0;
}
```

8.9.3 内存泄漏

```
// ? 内存泄漏：分配了但没有释放
void leak()
{
    int *p = malloc(100 * sizeof(int));
    // 忘记 free(p);
}   // 函数返回后，p被销毁，但分配的内存没有释放!

// ? 正确做法
void noLeak()
{
    int *p = malloc(100 * sizeof(int));
    if (p != NULL)
    {
        // 使用...
        free(p);
    }
}
```

8.10 综合案例：动态数组排序

```
#include <stdio.h>
#include <stdlib.h>

// 冒泡排序
void bubbleSort(int *arr, int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```c
int main()
{
    int n;
    printf("请输入元素个数: ");
    scanf("%d", &n);

    // 动态分配
    int *arr = (int*)malloc(n * sizeof(int));
    if (arr == NULL) { printf("分配失败\n"); return 1; }

    printf("请输入%d个整数: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    bubbleSort(arr, n);

    printf("排序后: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}
```

8.11 本章小结

知识点	要点
指针定义	`int *p = &a;` p存a的地址，*p取a的值
& 和 *	`&`取地址，`*`通过地址取内容
指针运算	`p+1`偏移`sizeof(类型)`字节
数组名	地址常量，等于首元素地址
指针与数组	`arr[i]` 等价于 `*(arr+i)`
指针传参	可实现函数内修改外部变量
动态内存	`malloc/free` 运行时分配/释放
野指针	指向已释放内存的指针，free后应置NULL
内存泄漏	分配但未释放，程序长期运行会耗尽内存

8.12 课后作业

基础练习

1. 选择题

(1) 有定义 `int a = 5, *p = &a;` 则 `*p + 1` 的值是（　）：

A. 6　　B. a的地址+1　　C. 5　　D. 不确定

(2) `int arr[10];` 则 `arr` 的值为（　）：

A. 数组全部元素　　B. 数组第一个元素的值

C. 数组首元素的地址　　D. 10

(3) 以下能正确进行字符串赋值的是（　）：

A. `char s[5] = {"ABCDE"};`　　B. `char s[5] = {'A','B','C','D','E'};`

C. `char *s = "ABCDE";`　　D. 以上都对

2. 程序阅读

```c
int a[] = {1, 2, 3, 4, 5};
int *p = a;
printf("%d\n", *(p+2));  // 输出?
```

编程练习

3. 指针实现字符串长度

编写函数 `int strLen(char *str)` 用指针实现求字符串长度（不用 strlen）。

4. 指针实现数组逆序

编写函数 `void reverse(int *arr, int n)` 用指针操作实现数组逆序。

5. 动态数组统计

从键盘输入n个整数（n由用户输入），动态分配内存存储，然后统计最大值、最小值、平均值并排序输出。

6. 字符串比较

编写函数 `int strCmp(const char *s1, const char *s2)` 用指针实现字符串比较。

7. 动态二维数组

用指针动态创建一个m×n的二维数组，输入并打印。

> **学习建议**：指针是C语言的难点，也是区分"会不会C"的分水岭。画内存图是最有效的学习方法——把地址、值、指向关系画出来，一目了然。